Comment GMIC analyse vos mammographies

Guide visuel de l'inférence — de l'image prétraitée au score de cancer

Pourquoi une étape d'inférence séparée ?

Le prétraitement (`make preprocess`) produit des images propres et uniformes dans `cropped_images/`. L'inférence (`make infer`) prend ces images et produit un **score de probabilité de cancer** pour chacune, en faisant tourner 5 modèles GMIC en parallèle et en moyennant leurs prédictions (**vote d'ensemble**).

Étape 1 — Préparer l'image pour le modèle

Avant d'entrer dans le réseau, chaque image subit 3 transformations appliquées à la volée (pas stockées sur disque) :

```
# inference.py · ligne 45
def _load_and_prepare(image_path, view):
    # 1. Lecture en float32 - ligne 52
    img = np.array(imageio.imread(image_path)).astype(np.float32)

    # 2. Flip horizontal des vues droites (R-CC, R-MLO) - ligne 53-54
    #    Le modèle a été entraîné avec tous les seins orientés vers la gauche.
    #    Sans ce flip, les vues droites seraient "à l'envers" pour GMIC.
    if view in {"R-CC", "R-MLO"}:
        img = np.fliplr(img).copy()

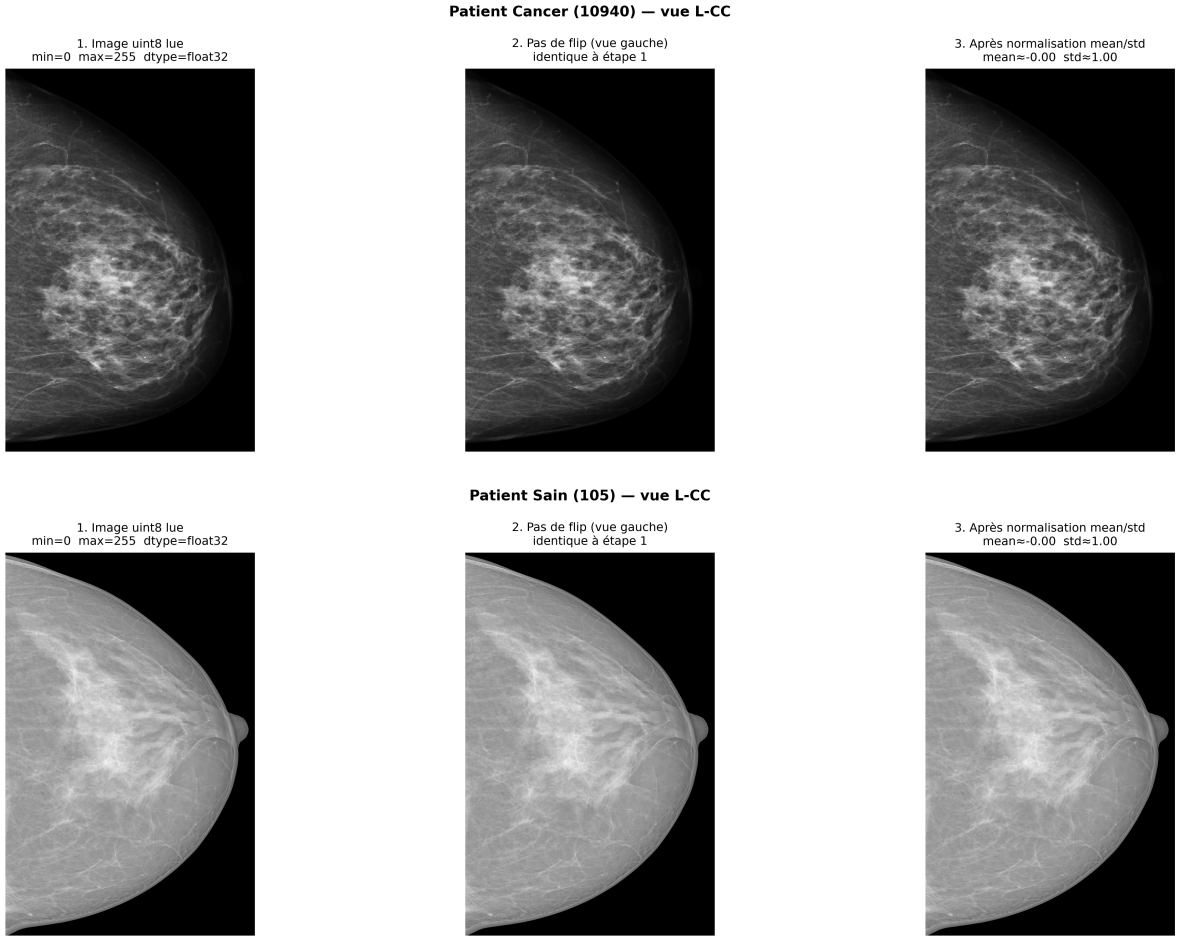
    # 3. Normalisation mean/std - lignes 55-56
    #    Ramène les pixels à une distribution ~N(0, 1).
    #    Rend le modèle insensible aux variations de luminosité entre scanners.
    img -= img.mean()
    img /= max(img.std(), 1e-5)
```

```
    return img
```

Ces 3 étapes sont la seule préparation nécessaire. Le modèle reçoit ensuite un tenseur `(1, 1, 2944, 1920)` float32 — une image, un canal (niveaux de gris).

Visualisation : avant et après normalisation

```
/tmp/ipykernel_52216/2461728781.py:18: DeprecationWarning: Starting with ImageIO v3 the beha
  img_raw = np.array(imageio.imread(img_path)).astype(np.float32)
```



Étape 2 — Convertir l'image en tenseur PyTorch

Un réseau de neurones PyTorch ne "voit" pas des fichiers PNG. Il reçoit des **tenseurs** — des tableaux NumPy multidimensionnels emballés dans un objet PyTorch qui sait comment faire tourner les calculs sur GPU ou CPU.

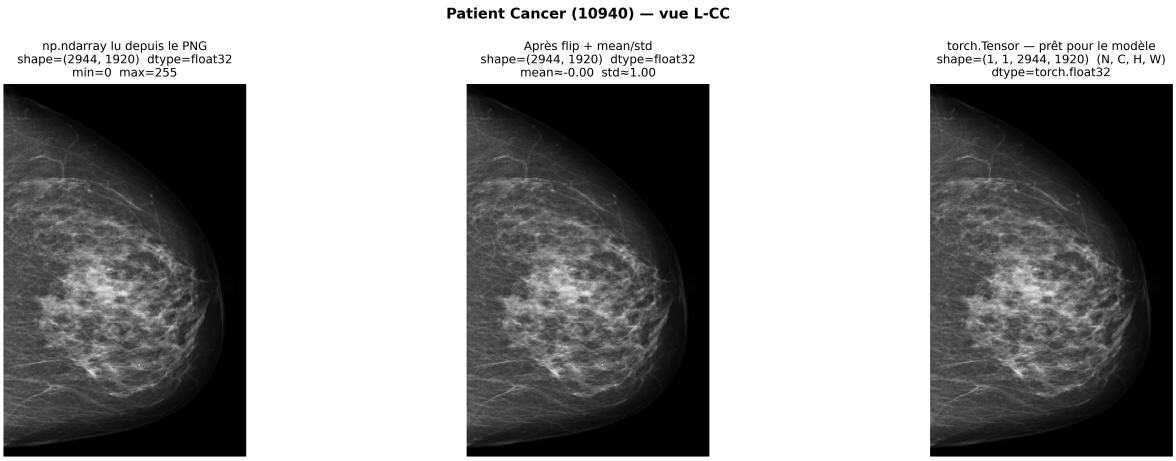
La conversion se fait en deux lignes :

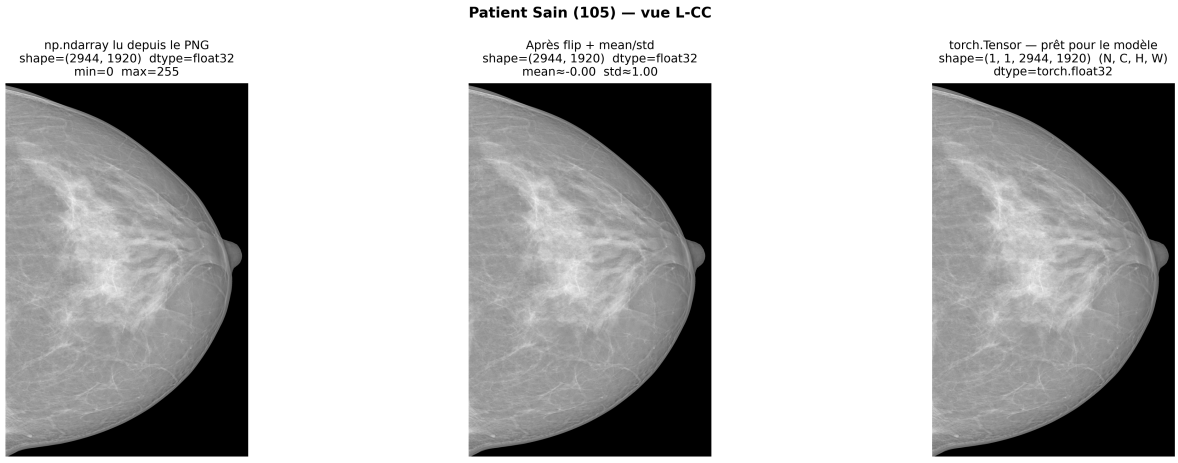
```
# inference.py · ligne 112
# img est un np.ndarray (2944, 1920) float32, déjà normalisé
tensor = torch.Tensor(img[np.newaxis, np.newaxis])
#
#                           ajout dim    ajout dim
#                           "batch"      "canal"
# résultat : tenseur de forme (1, 1, 2944, 1920)
#                 batch size = 1 (on envoie 1 image à la fois)
#                     canaux = 1 (image en niveaux de gris, pas RGB)
```

PyTorch attend toujours des tenseurs en format $\mathbf{N}\times\mathbf{C}\times\mathbf{H}\times\mathbf{W}$: N = nombre d'images dans le batch, C = canaux couleur, H = hauteur, W = largeur. Une photo couleur aurait C=3 (R, G, B). Ici C=1 car une mammographie est en niveaux de gris.

Visualisation : du tableau NumPy au tenseur

```
/tmp/ipykernel_52216/2617485687.py:19: DeprecationWarning: Starting with ImageIO v3 the beha
  img_raw = np.array(imageio.imread(img_path)).astype(np.float32)
```





Étape 3 — Instancier et charger le modèle

Un modèle PyTorch est une **classe Python** héritant de `nn.Module`. L'instancier crée la structure du réseau (toutes les couches) avec des poids **aléatoires**. Pour avoir les vrais poids appris pendant l'entraînement, il faut les charger depuis un fichier `.p` (pickle de state_dict) :

```
# inference.py · lignes 129-133
# 1. Créer la structure vide du modèle avec ses hyperparamètres
model = gmic_module.GMIC(params)
# À ce stade, model existe mais ses poids sont aléatoires - inutilisable.

# 2. Charger les poids pré-entraînés depuis le fichier .p
state_dict = torch.load("GMIC/models/sample_model_1.p", map_location="cpu")
model.load_state_dict(state_dict, strict=False)
# Maintenant model contient les poids appris sur INbreast/CBIS-DDSM.

# 3. Passer en mode évaluation (désactive dropout, batch norm en mode train)
model.eval()

# 4. Inférence sans calculer les gradients (plus rapide, moins de mémoire)
with torch.no_grad():
    output = model(tensor)   # tensor shape : (1, 1, 2944, 1920)
    # output shape : (1, 2)  →  [score_bénin, score_malin]
```

`strict=False` : les poids du fichier `.p` ont été entraînés avec une version légèrement différente de l'architecture. `strict=False` permet de charger les poids qui correspondent et d'ignorer les autres.

`torch.no_grad()` : pendant l'entraînement, PyTorch mémorise toutes les opérations pour calculer les gradients (backpropagation). À l'inférence on n'en a pas besoin — ce contexte désactive ce mécanisme et divise par ~2 la consommation mémoire.

Étape 4 — Pourquoi 5 modèles ? Le vote d'ensemble

On charge et fait tourner **5 modèles** distincts, puis on **moyenne** leurs prédictions :

```
# inference.py · lignes 127-147
all_dfs = []
for idx in ["1", "2", "3", "4", "5"]:
    # Chaque modèle a été entraîné avec un percent_t différent
    # (seuil de sélection des patches : 0.02, 0.03, 0.03, 0.05, 0.10)
    params["percent_t"] = PERCENT_T_DICT[idx]

    model = gmic_module.GMIC(params)
    model.load_state_dict(torch.load(f"sample_model_{idx}.p", map_location="cpu"), strict=
    model.eval()

    # Faire tourner ce modèle sur toutes les images
    df = _run_one_model(model)
    all_dfs.append(df)

# Moyenne des 5 scores pour chaque image
pred_df = pd.concat(all_dfs).groupby("image_index").apply(
    lambda r: pd.Series({"malignant_pred": np.nanmean(r["malignant_pred"])})
)
```

Pourquoi ça marche ? Chaque modèle fait des erreurs différentes (ils ne voient pas tous les cas difficiles de la même façon). En moyennant, les erreurs individuelles se compensent et la prédiction finale est plus robuste qu'un seul modèle — c'est le principe du **vote d'ensemble**.

Ce que retourne le modèle sur vos images

```
/tmp/ipykernel_52216/417720359.py:30: DeprecationWarning: Starting with ImageIO v3 the behavi
  img = np.array(imageio.imread(img_path)).astype(np.float32)
```

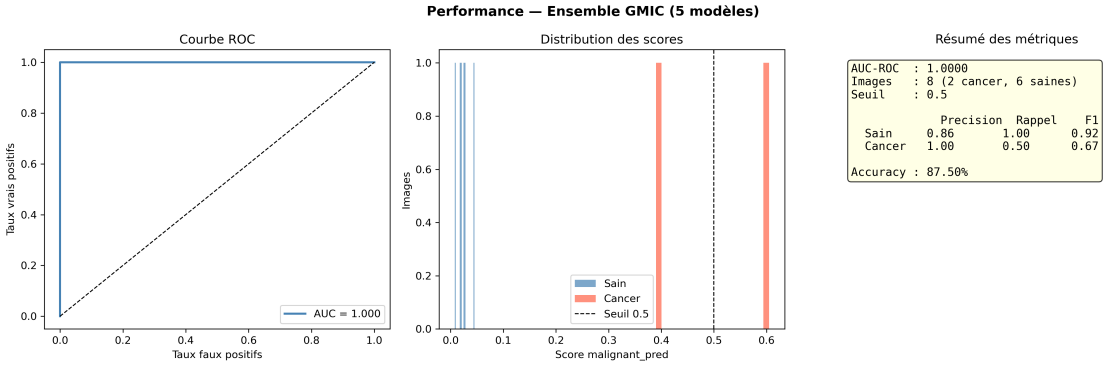
Patient	Vue	score bénin	score malin	→ verdict
Cancer	L-CC	0.1428	0.0359	sain
	L-MLO	0.1025	0.0304	sain
	R-CC	0.4385	0.3389	sain
	R-MLO	0.7017	0.7091	CANCER
Sain	L-CC	0.0876	0.0125	sain
	L-MLO	0.2220	0.0345	sain
	R-CC	0.2111	0.0372	sain
	R-MLO	0.1748	0.0345	sain

Résultats

8 images analysées — 2 cancer, 6 saines

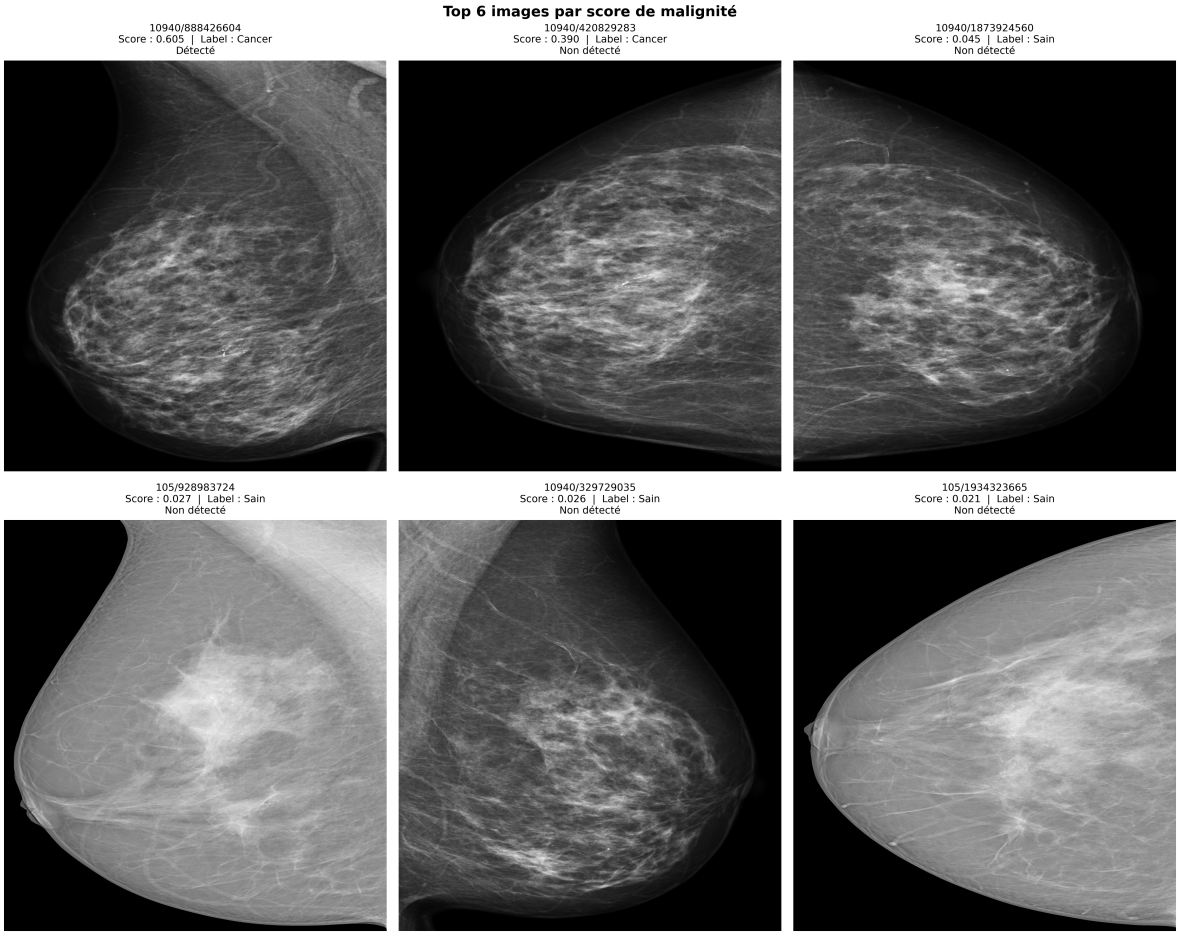
	image_index	malignant_pred	benign_pred	malignant_label
7	10940/888426604	0.6048	0.6360	1.0
6	10940/420829283	0.3899	0.4487	1.0
4	10940/1873924560	0.0451	0.1569	0.0
3	105/928983724	0.0272	0.1789	0.0
5	10940/329729035	0.0257	0.0860	0.0
1	105/1934323665	0.0209	0.1798	0.0
0	105/1232990271	0.0181	0.1834	0.0
2	105/397491913	0.0080	0.1218	0.0

Métriques et courbe ROC



Cas les plus suspects

Les images avec le score `malignant_pred` le plus élevé. Bordure **verte** = prédiction correcte, **rouge** = erreur.



Relancer

```
# Pipeline complet
make run INPUT_DIR=data/sample/train_images OUTPUT_DIR=output/sample

# Étapes séparées
make preprocess INPUT_DIR=data/sample/train_images OUTPUT_DIR=output/sample
make infer      OUTPUT_DIR=output/sample
```